

CSE 574 - Computational Complexity

Zhengjie Min

September 9, 2026

Abstract

This is a template for TCS (and other kind of) lecture notes.

Contents

I	Introduction & Background Review	2
1	The Beginning	3
1.1	Definition of Computational Theory	3
1.2	What is Complexity About?	4
2	Time and Non-deterministic Time	5
2.1	Languages and Problems	5
2.2	Classes of Complexity	5
2.2.1	Deterministic Time Complexity	5
2.2.2	Non-deterministic Time Complexity	5
A	Algebra	8
B	Analysis	9

Part I

Introduction & Background Review

Chapter 1

The Beginning

The start of computational theory can be traced back to the 1930s, when mathematicians and logicians were trying to formalize the notion of computation and algorithms. The Church-Turing thesis, proposed independently by Alonzo Church and Alan Turing, laid the foundation for the study of computational complexity.

Theorem 1.0.1 (Church-Turing Thesis). Every effectively calculable function is computable by a Turing machine.

Then mathematicians and computer scientists began to explore the limits of computation, with such an open thesis:

Problem 1.0.1 (Extended (Quantum) Church-Turing Thesis). Every physically realizable model of computation can be simulated by a (quantum) Turing machine efficiently.

1.1 Definition of Computational Theory

Remark 1.1.1. Computational theory deals with how much resources we need to solve a computational problem.

Resources can be many things. For example:

- Time
- Memory / Space
- Samples Complexity (how many samples we need to distinguish between distributions)
- Communication Rounds
- Oracle Calls
- Randomness
- Power
- Quantum Entanglement
- Number of Parallel Workers
- ...

1.2 What is Complexity About?

In CS Theory, people do research in algorithms and complexity. People might lean on one side or the other, but they are closely related. In a word:

Remark 1.2.1. Algorithms are about the upper bound of a problem, while complexity is about the lower bound of a problem.

Chapter 2

Time and Non-deterministic Time

2.1 Languages and Problems

In computational complexity, we talk about languages as how we think about computational problems.

Definition 2.1.1. A **language** L is a set of strings that $L \subseteq \Sigma^*$, where Σ is a finite alphabet (often $\{0, 1\}$ bits.)

Example 2.1.1. An algorithm $\text{PRIMES}(n)$ can decide whether a number n is prime or not. The language of this problem is:

$$\text{PRIMES} = \{\langle x \rangle : x \text{ is prime}\}.$$

$\langle x \rangle$ is the encoding of x in binary.

2.2 Classes of Complexity

2.2.1 Deterministic Time Complexity

Definition 2.2.1. $L \in \text{DTIME}(T(n))$ if there's a deterministic Turing machine M that decides L in time $O(T(n))$, where n is the size of the input. $P := \bigcup_{c \geq 1} \text{DTIME}(n^c)$ is the class of problems that can be solved in polynomial time.

Researchers have found a polynomial time algorithm for the PRIMES problem in 2002, so

$$\text{PRIMES} \in P.$$

Example 2.2.1.

$$\text{CONN} = \{x : x \text{ is the adjacency matrix of a connected graph}\}$$

$\text{CONN} \in P$ because we can use DFS or BFS to check connectivity in polynomial time.

2.2.2 Non-deterministic Time Complexity

Definition 2.2.2. A **non-deterministic Turing machine** is a Turing machine that can make non-deterministic choices at each step. It can be thought of as a machine that can explore multiple computation paths simultaneously, or a machine that takes different branches of operations in different universes.

For instance, assume a certain state. In this state, the transition function for a deterministic TM might be just $(0, L, 1)$ and $(1, R, 1)$, while a non-deterministic TM might have $(0, R, 1)$, $(0, L, 0)$, and $(1, L, 1)$. When the machine meets a 0, it may arbitrarily choose to go left or right.

Definition 2.2.3. $L \in \text{NTIME}(T(n))$ if there's a non-deterministic Turing machine M that decides L in time $O(T(n))$ (regardless of which universe.) Here 'decides' means that if $x \in L$, there exists a computation path that accepts x ; if $x \notin L$, all computation paths reject x .

Definition 2.2.4. $L \in \text{co-NTIME}(T(n)) \Leftrightarrow \bar{L} \in \text{NTIME}(T(n))$.

Definition 2.2.5. $\text{NP} := \bigcup_{c \geq 1} \text{NTIME}(n^c)$ is the class of problems that can be solved in polynomial time by a non-deterministic Turing machine.

Remark 2.2.1. We normally consider NP to be 'verifiable' in polynomial time, using a 'certificate' u . Consider a non-deterministic TM that accepts x in a certain path. The sequence of non-deterministic choices made by the TM can be encoded as a string u . Then we can construct a deterministic TM $M(., .)$ that takes x and u as input, and verify whether $x \in L$ by checking if there exists a certificate u such that $M(x, u)$ accepts in polynomial time. This can also explain why the certificate is required to be in size $O(T(n))$.

In mathematical terms:

$$x \in L \Leftrightarrow \exists u \in \{0, 1\}^{O(T(n))} \text{ s.t. } M(x, u) = 1.$$

Then for NP, $T(n) = \text{poly}(n)$.

Example 2.2.2.

- Independent Set.

$$\text{IND-SET} = \{ \langle G, k \rangle : \exists S \subseteq V(G) \text{ s.t. } |S| \geq k \text{ and } \forall u, v \in S, \{u, v\} \notin E(G) \}$$

Independent Set is in NP because we can verify a solution by manually checking the edges in the graph.

- Traveling Salesman Problem (TSP) is about many cities, with d_{ij} being the distance between city i and city j .

The problem is whether there exists a closed circuit that visits each city exactly once, with total distance at most k .

TSP is in NP because we can calculate the distance of a circuit in polynomial time.

- Linear Programming: $A\vec{x} \leq \vec{b}$. Is there a solution $\vec{x}$?

LP is trivially NP, but people later found that LP is also in P.

- Graph Isomorphism: Given two adjacency matrices, do they define the same graph up to permutation of vertices?

Graph Isomorphism is in NP because we can verify a solution by checking if such permutation of vertices gives the same adjacency matrix.

- Factoring.

$$\text{FACTORING} = \{ \langle N, k \rangle : N \text{ has a factor } \leq k \}$$

Equivalently, it can also decide $\{ \langle N, L, R \rangle : N \text{ has a prime factor between } L \text{ and } R \}$.

Factoring is in NP as we can just divide N by a certificate.

Appendix

Appendix A

Algebra

Theorem A.0.1. For any polynomial $A \in \mathbb{C}[X]$ of degree $n - 1$, any point set $\{p_0, \dots, p_{n-1}\}$ of size n , $\{(p_i, A(p_i))\}_{i=0}^{n-1}$ determines $A(x)$ and vice versa.

Theorem A.0.2 (Fundamental Theorem of Algebra). Every non-constant single-variable polynomial with complex coefficients has at least one complex root.

In math terms, for $A(x) = \sum_{i=0}^{n-1} a_i x^i$, either $\exists x \in \mathbb{C}$ such that $A(x) = 0$, or $a_0 = a_1 = \dots = a_{n-1} = 0$.

Appendix B

Analysis

Theorem B.0.1 (Master Theorem (simplified)). Let the recurrence relation be:

$$T(n) = aT\left(\frac{n}{b}\right) + n^d$$

where $a \geq 1$ is the number of subproblems, $b > 1$ is the factor by which the problem size is reduced, and $d \geq 0$. The asymptotic bound of $T(n)$ is given by:

1. If $\log_b a > d$, then $T(n) = O(n^{\log_b a})$.
2. If $\log_b a = d$, then $T(n) = O(n^d \log n)$.
3. If $\log_b a < d$, then $T(n) = O(n^d)$.

Theorem B.0.2 (Master Theorem). Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence:

$$T(n) = aT(n/b) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

- **Case 1:** If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
- **Case 3:** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n (Regularity Condition), then $T(n) = \Theta(f(n))$.